
PLATO · AI-ENABLED QA & SDET TESTING

Risk-Based Regression Agent

*Installation and user guide — prerequisites, inputs,
parameters and daily operation*

Version Proof of Concept, Phase 3

Applies to plato-risk-regression-poc

Audience QA engineers, SDETs, and evaluators

Date September 2026

Contents

Part 1 Understanding the agent

- 1.1 The problem it solves
- 1.2 The two flows
- 1.3 What it does — and will not do
- 1.4 How a test gets selected

Part 2 Prerequisites

- 2.1 Machine and software
- 2.2 Optional accounts
- 2.3 What you need from your own project

Part 3 Installation

- 3.1 Install in five steps
- 3.2 Verify the install

Part 4 Inputs — every data source in detail

- 4.1 Overview of the eleven sources
- 4.2 Flow 1 files
- 4.3 Flow 2 files
- 4.4 Phase 3 signal files
- 4.5 The test map
- 4.6 Live Jira and Confluence

Part 5 Parameters — every setting

- 5.1 How settings are resolved
- 5.2 Environment variables
- 5.3 Command-line options
- 5.4 The threshold, explained

Part 6 Running the agent

- 6.1 Commands by task
- 6.2 Schedule profiles
- 6.3 What gets written where

Part 7 Using it on your own project

Part 8 Troubleshooting

Appendix Repository map

Part 1 • Understanding the agent

Read this part if you are new to the tool. If you only need to install and run it, skip to Part 2.

1.1 The problem it solves

Running an entire regression suite on every release is slow and expensive. The usual alternative — a senior QA picking a handful of tests by instinct — is fast but leaves no record of *why* a test was skipped. If something ships broken, nobody can reconstruct the decision.

This agent chooses the tests worth running, runs only those, and records a reason for every choice, including every skip. The output is a short, justified, auditable test run.

1.2 The two flows

The agent answers two different questions, each from its own signal. They share one engine and one self-heal step; only the input and the scoring model differ.

	FLOW 1 — DEFECT HISTORY	FLOW 2 — CHANGE DRIVEN
Question	What has broken before?	What just changed?
Direction	Looks backwards	Looks forwards
Reads	Past defects from Jira, a defect CSV, and code churn	This release's user stories, bug fixes, requirement changes, and churn
Model	$\text{risk} = \text{severity} \times \text{count} \times \text{recency} \times (1 + \text{churn})$, then $\times 1.6$ if the defect was reopened	$\text{impact} = (2 \cdot \text{features} + 2 \cdot \text{fixes} + 3 \cdot \text{requirement changes}) \times (1 + \text{churn})$
Command	<code>python run_poc.py --flow 1</code>	<code>python run_poc.py --flow 2</code>

On the bundled demo data the two flows deliberately land on different test sets — Flow 1 on `login` and `checkout`, Flow 2 on `payments` and `checkout`. That contrast is the point: two signals, two answers, one engine.

1.3 What it does — and will not do

IT DOES	IT WILL NOT
Rank components by risk or impact and select their tests	Invent a test where none exists
Run only the selected tests with pytest	Write to your main branch
Repair a stale test and open it as a pull request for review	Edit application source to make a failing test pass
File a real Jira defect when the application is genuinely wrong	Merge anything without a human approving it
Log every decision, run or skip, with its reason	Hide a low-confidence result behind a confident number

THE RULE THAT CANNOT BE TURNED OFF

When a selected test fails, the agent decides whether the *test* is stale or the *application* is broken. A stale test is fixed via a pull request. A broken application produces a Jira defect and nothing else — the agent never patches product code to turn a red test green.

1.4 How a test gets selected

Selection is deterministic, not a judgement call by a language model. The steps are always the same:

1. **Read** every configured source and normalise it into one shape.
2. **Score** each component with the flow's formula (Part 1.2).
3. **Compare** each score against the threshold (default `5.0`, see 5.4).
4. **Select** the test files mapped to components at or above the threshold, via `tests/test_map.json`.
5. **Run** only those files, and report what was skipped and why.

The language model is used for the surrounding work — summarising defects in plain language, diagnosing a failure, drafting a defect report — not for picking the tests.

Part 2 • Prerequisites

Everything you need before installing. Nothing here is unusual; most teams already have all of it.

2.1 Machine and software

REQUIREMENT	MINIMUM	NOTES
Operating system	Windows 10, macOS 12, or Linux	Developed and demonstrated on Windows with PowerShell. No OS-specific code.
Python	3.9 or newer	Check with <code>python --version</code> . On macOS/Linux you may need <code>python3</code> .
pip	bundled with Python	<code>python -m pip --version</code>
Disk	~50 MB	Plus roughly 120 MB if you install the optional Playwright browser.
Network	Only for live mode	Everything else runs fully offline from the bundled CSVs.
git (optional)	any recent	Only needed for <code>PR_MODE=1</code> and <code>--pr-check</code> .
gh (optional)	any recent	GitHub CLI. Only needed to open a real pull request automatically.

Python packages

Three, installed by `requirements.txt`:

PACKAGE	VERSION	NEEDED FOR
<code>anthropic</code>	≥ 0.40	The live agent runs: <code>--flow</code> and <code>--demo</code> . Not used by the offline commands.
<code>pytest</code>	≥ 7.0	Actually executing the selected tests. Required.
<code>requests</code>	≥ 2.31	Live Jira and Confluence reads, and filing defects.
<code>playwright</code>	≥ 1.40	<i>Optional</i> . Only for the live browser tests against saucedemo.com.

2.2 Optional accounts

The agent runs completely without any account, reading the bundled CSV files. You only need these to enable the matching feature.

ACCOUNT	ENABLES	WHERE TO GET IT
Anthropic API key	The live agent runs and self-healing (<code>--flow</code> , <code>--demo</code>)	platform.claude.com. Billed per token; one full flow run costs a few cents.
Atlassian API token	Live Jira defect reads, live Confluence requirements, and filing defects	id.atlassian.com → Security → API tokens. One token covers both Jira and Confluence.

2.3 What you need from your own project

To point the agent at your product rather than the bundled demo app, you need two mappings. Most teams have both already, in some form.

1. A component vocabulary

An agreed list of the parts of your application — `checkout`, `payments`, `search`, and so on. Every defect, story and test is labelled with one of these. It does not need to be perfect; it needs to be consistent.

2. A test-to-component map

Which test file covers which component. This is `tests/test_map.json` (see 4.5). If your test files are already named after the areas they cover, this is a short file to write.

HONEST LIMITATION

The agent can only select tests that exist and are mapped. Where a risky component has no test, it cannot conjure one — but it will tell you, which is often the more useful output. Run `python run_poc.py --coverage` to get that list ranked by risk.

Part 3 • Installation

Five steps, about five minutes. Commands are shown for Windows PowerShell; macOS and Linux differences are noted.

3.1 Install in five steps

1. Get the code

```
git clone https://github.com/shukkoorkassim/plato-risk-regression-poc.git
cd plato-risk-regression-poc
```

Or copy the project folder onto the machine and `cd` into it. Every command in this guide is run from the project root — the folder containing `run_poc.py`.

2. Create a virtual environment (recommended)

```
# Windows PowerShell
python -m venv .venv
.venv\Scripts\Activate.ps1

# macOS / Linux
python3 -m venv .venv
source .venv/bin/activate
```

This keeps the agent's packages away from your system Python. Skip it if your team has another convention.

3. Install the packages

```
python -m pip install -r requirements.txt
```

4. Create your settings file

```
# Windows
copy .env.example .env

# macOS / Linux
cp .env.example .env
```

Open `.env` and fill in only what you need. Every setting is optional except `ANTHROPIC_API_KEY`, and even that is only required for the two commands that call the language model. Part 5 documents every setting.

NEVER COMMIT .ENV

`.env` holds live credentials and is excluded by `.gitignore`. Do not rename it, do not copy it to a `.txt` file, and do not paste its contents into chat or a ticket.

5. Plant the demo state

```
python run_poc.py --reset
```

This restores the two deliberately planted bugs in the demo application and clears any leftover agent state. Run it before any demonstration, and again between Flow 1 and Flow 2 — the agent really does fix the code, so a second run would otherwise find nothing to repair.

3.2 Verify the install

One command checks everything on the machine, including live connections if you configured them:

```
python run_poc.py --selftest
```

It exits `0` when healthy, so it can be wired straight into CI. If you have not configured any credentials, it will confirm the offline path works, which is a valid installation.

Two further commands prove the pipeline end to end without needing an API key:

COMMAND	WHAT IT PROVES
<code>python run_poc.py --dry-run</code>	Both flows read, score, select and run — and the planted bugs fail, as they should. No API key, no network.
<code>python run_poc.py --showcase</code>	The whole story in ten narrated console phases. Add <code>--pace 1.0</code> to slow it for an audience, <code>--ascii</code> for older terminals.

Part 4 • Inputs — every data source in detail

What the agent reads, in what shape, and what happens when a source is missing.

4.1 Overview of the sources

Every source has a file form and, for some, a live form. When a live source is not configured, that source silently falls back to its file — the run never fails because a credential is absent.

SOURCE	USED BY	LIVE OPTION	PURPOSE
data/jira_export.csv	Flow 1	Jira (JIRA_LIVE=1)	Past defects with severity and resolution date
data/defects.csv	Flow 1	—	A second defect export; carries the reopened flag
data/git_churn.csv	Flow 1	—	How much each component has been edited
release_delta/stories.csv	Flow 2	Jira (CHANGE_LIVE=1)	Features delivered this release
release_delta/bugfixes.csv	Flow 2	Jira (CHANGE_LIVE=1)	Defects fixed this release
release_delta/requirements.csv	Flow 2	Confluence	Requirements that changed this release
release_delta/release_churn.csv	Flow 2	— (<i>always a file</i>)	Code churn for this release
data/business_value.csv	Signals	—	How important each component is commercially
data/sprint_history.csv	Signals	—	Spillover and carry-over per component
data/rca_notes.csv	Signals	—	Root causes, and whether they are systemic
tests/test_map.json	Both	—	Which test file covers which component

WHY CHURN IS ALWAYS A FILE

Churn comes from git history, not from a tracker, so there is no Jira or Confluence equivalent to read it from. In a real deployment you would generate this CSV from `git log` as a pipeline step.

4.2 Flow 1 files

data/jira_export.csv

COLUMN	TYPE	MEANING
key	text	Issue key, e.g. <code>SWAG-201</code>
component	text	Must match a component name used in the test map
severity	enum	<code>blocker</code> , <code>critical</code> , <code>major</code> , <code>minor</code>
summary	text	Free text; shown in the agent's summary step
resolved	date	<code>YYYY-MM-DD</code> . Drives the recency weighting

data/defects.csv

Same idea, with two differences: the date column is `fixed_date`, and there is a `reopened` column.

COLUMN	TYPE	MEANING
id	text	Defect id, e.g. <code>DEF-11</code>
component	text	As above
severity	enum	As above
summary	text	Free text
fixed_date	date	<code>YYYY-MM-DD</code>
reopened	boolean	<code>true</code> multiplies this defect's weight by 1.6

data/git_churn.csv

COLUMN	TYPE	MEANING
commit	text	Short SHA
component	text	Which component the commit touched
message	text	Commit message
date	date	<code>YYYY-MM-DD</code>

Churn is counted as commits per component, then applied as $(1 + \text{churn}/10)$.

4.3 Flow 2 files

All four live in `data/release_delta/`.

FILE	COLUMNS
<code>stories.csv</code>	<code>key, component, summary, points</code>
<code>bugfixes.csv</code>	<code>key, component, severity, summary, fixed_date</code>
<code>requirements.csv</code>	<code>key, component, summary, changed_date</code>
<code>release_churn.csv</code>	<code>commit, component, message, files_changed, date</code>

Requirement keys are conventionally `REQ-###`; this is what the Confluence reader looks for when live (4.6).

4.4 Phase 3 signal files

These three sharpen the score but are not required. They currently ship with placeholder numbers — the `source` column says `PLACEHOLDER`, and the agent announces this on every run rather than presenting invented figures as real.

FILE	COLUMNS	REPLACE WITH
<code>business_value.csv</code>	<code>component, criticality, monthly_sessions, revenue_share_pct, source</code>	Your analytics export
<code>sprint_history.csv</code>	<code>sprint, component, carried_over, spillover_days, reopened_in_sprint, source</code>	A Jira sprint report export
<code>rca_notes.csv</code>	<code>defect_id, component, root_cause, systemic, note, source</code>	Your RCA field, once populated

Inspect what these currently contribute with `python run_poc.py --signals`.

4.5 The test map

`tests/test_map.json` is the single most important file when adopting the agent. It is a flat object: test file path to component name.

```
{
  "tests/test_login.py": "login",
  "tests/test_inventory.py": "inventory",
  "tests/test_cart.py": "cart",
  "tests/test_checkout.py": "checkout",
  "tests/test_search.py": "search",
  "tests/test_payments.py": "payments"
}
```

Rules: one component per test file; the component names here must match those used in your defect and change data exactly; a test file absent from this map can never be selected.

CHECK IT BEFORE YOU TRUST IT

`python run_poc.py --check-data` reports components with no test and tests with no component. Run it after every change to the map.

4.6 Live Jira and Confluence

Flow 1 — defects from Jira

Set `JIRA_LIVE=1` plus the three `ATLASSIAN_*` values. The default query targets this POC's seeded issues:

```
project = <JIRA_PROJECT_KEY> AND labels = defect-history ORDER BY resolved DESC
```

For a real project, override it with the resolved-bugs basis:

```
JIRA_JQL=issuetype = Bug AND statusCategory = Done AND resolved >= -180d
```

Component and severity are read from the issue's **Components** and **Priority** fields. Where those are empty — common in team-managed projects — the agent falls back to the issue's **labels**.

Flow 2 — delta from Jira and Confluence

Set `CHANGE_LIVE=1`. Features and bug fixes come from two label-based queries; requirement changes come from a Confluence page, identified either by `CONFLUENCE_PAGE_ID` or by `CONFLUENCE_SPACE_KEY` plus `CONFLUENCE_PAGE_TITLE`. Any table row on that page keyed `REQ-###` is picked up.

```
python scripts/check_jira.py      # verify the Flow 1 connection
python scripts/check_changes.py   # verify the Flow 2 connections
```

Part 5 • Parameters — every setting

Twenty-one environment variables and thirty command-line options, with their defaults.

5.1 How settings are resolved

A setting can come from three places. Later entries win:

1. The built-in default in the code.
2. `.env` in the project root.
3. A real environment variable in your shell.

A few settings also have a command-line flag, which wins over all three for that run — `--threshold` is the main one.

All boolean settings accept `1`, `true`, `yes` or `on`. Anything else, including absent, counts as off.

5.2 Environment variables

Language model

VARIABLE	DEFAULT	EFFECT
<code>ANTHROPIC_API_KEY</code>	<code>none</code>	Required by <code>--flow</code> and <code>--demo</code> . Unused by every offline command.
<code>ANTHROPIC_MODEL</code>	<code>claude-sonnet-5</code>	Which model the agent reasons with.

Selection

VARIABLE	DEFAULT	EFFECT
<code>RISK_THRESHOLD</code>	<code>5.0</code>	The score a component must reach for its tests to run. Applies to both flows. See 5.4.

Atlassian connection

VARIABLE	DEFAULT	EFFECT
<code>ATLASSIAN_SITE</code>	<code>none</code>	e.g. <code>your-site.atlassian.net</code> . A leading <code>https://</code> is stripped automatically.
<code>ATLASSIAN_EMAIL</code>	<code>none</code>	The account the API token belongs to.
<code>ATLASSIAN_TOKEN</code>	<code>none</code>	API token. Covers both Jira and Confluence.
<code>JIRA_PROJECT_KEY</code>	<code>KAN</code>	Used by every default query and when filing defects.

Flow 1 — live defects

VARIABLE	DEFAULT	EFFECT
JIRA_LIVE	off	On: read defects from Jira. Off, or credentials missing: read data/jira_export.csv .
JIRA_JQL	<i>POC query</i>	Override the defect query entirely.

Flow 2 — live release delta

VARIABLE	DEFAULT	EFFECT
CHANGE_LIVE	off	On: features and fixes from Jira, requirements from Confluence. Churn always stays a CSV.
CONFLUENCE_PAGE_ID	<i>none</i>	The requirements page, by id.
CONFLUENCE_SPACE_KEY	<i>none</i>	Alternative to the id: the space.
CONFLUENCE_PAGE_TITLE	<i>none</i>	Alternative to the id: the exact page title.
JIRA_FEATURE_JQL	<i>label query</i>	Override the feature query.
JIRA_BUGFIX_JQL	<i>label query</i>	Override the bug-fix query.

Defect filing

VARIABLE	DEFAULT	EFFECT
REPORT_BUGS	off	On: the agent creates a real Jira issue when it finds a genuine application bug. Leave off when rehearsing a demo.
JIRA_DEFECT_ISSUETYPE	Task	Issue type to create. Task is the default because many team-managed projects have no Bug type.
JIRA_BUG_ISSUETYPE	Task	Legacy alias, read only if the above is unset.
JIRA_ASSIGNEE_EMAIL	ATLASSIAN_EMAIL	Who filed defects are assigned to.

Governance

VARIABLE	DEFAULT	EFFECT
APPROVE_ALL	off	On: auto-approves all four human gates. Off: each gate blocks and prints what it is waiting for.
PR_MODE	off	On, with git and gh available: a test fix opens a real branch and pull request. Off: the proposed file is written to _proposed_fixes/ . Never writes to main either way.

The four gates are: data quality reviewed, risk ratings reviewed, defect assignment approved, and test-fix merge approved.

5.3 Command-line options

All are options of `run_poc.py`. Running it with no arguments prints the built-in help.

Everyday

OPTION	WHAT IT DOES
<code>--selftest</code>	Checks everything on this machine, live connections included. Exits 0 when healthy.
<code>--dry-run</code>	Both flows, deterministic, no API key. Shows read → score → select → run.
<code>--report</code>	Both flows side by side: ranked, selected, filtered, executed.
<code>--flow 1 2 both combined</code>	The live agent, with self-healing. Needs an API key.
<code>--reset</code>	Re-plants the demo bugs and clears agent state.
<code>--threshold N</code>	Override the selection cut-off for this run.
<code>"free text"</code>	A quoted final argument overrides the agent's built-in task, e.g. <code>--flow 1 "Only the last 90 days."</code>

Demonstration

OPTION	WHAT IT DOES
<code>--showcase</code>	The full story in ten console phases, offline.
<code>--pace N</code>	With <code>--showcase</code> : seconds between beats. Try <code>1.0</code> when presenting.
<code>--ascii</code>	With <code>--showcase</code> : plain characters for older consoles.
<code>--demo</code>	The two-path self-heal: fixes a stale test, files a real application defect.

Quality and evaluation

OPTION	WHAT IT DOES
<code>--check-data</code>	Data-quality check on the test-to-component mapping.
<code>--coverage</code>	Risk areas that have no test, ranked by score.
<code>--eval</code>	The fixed evaluation set: expected versus actual selection. Run after any model change.
<code>--benchmark</code>	Accuracy against executed ground truth: recall, precision, F1, agreement.
<code>--per-bug</code>	With <code>--benchmark</code> : a row per seeded bug.
<code>--build-truth</code>	Rebuilds the ground truth by running the suite once per seeded bug. Slow.
<code>--compare-human</code>	The agent's selection against human labels.
<code>--csv PATH</code>	Use a different corpus for the two commands above.

<code>--human PATH</code>	Use a different human-label file.
---------------------------	-----------------------------------

Signals, governance and status

OPTION	WHAT IT DOES
<code>--signals</code>	What the Phase 3 signals currently contribute, and which are placeholders.
<code>--arch</code>	Architectural risk: fan-in and fan-out, boundaries, clustering.
<code>--path DIR</code>	With <code>--arch</code> : the package directory to analyse.
<code>--gates</code>	The approval gates and the write policy.
<code>--feedback</code>	What past runs and reviews have taught the agent.
<code>--feedback-demo</code>	Seeds the feedback store with a worked example.
<code>--pr-check</code>	Fast smoke check for a pull request.
<code>--base REF</code>	With <code>--pr-check</code> : the git ref to diff against.
<code>--schedule pr daily sprint quarterly</code>	Run under one schedule profile. See 6.2.
<code>--schedules</code>	All four profiles compared side by side.
<code>--status</code>	Status of all 21 requirements from the review.
<code>--gaps</code>	With <code>--status</code> : unfinished items only.
<code>--verify</code>	Runs the command behind every requirement and reports pass or fail.
<code>--deck-status</code>	What the deck claims versus what the code does.

5.4 The threshold, explained

This is the one number worth tuning. A component's tests run when its score reaches the threshold.

Default	5.0
Lower it	More tests run • fewer misses • less time saved
Raise it	Fewer tests run • more time saved • more risk of a miss

Set it for one run with `--threshold 3.0`, or permanently with `RISK_THRESHOLD` in `.env`. Change it, then re-run `-eval` and `--benchmark` to see what the change cost you in recall before adopting it.

DO NOT OPTIMISE FOR THE SKIP RATE

A high threshold produces an impressive "percentage skipped" and can quietly discard the one test that catches the bug. Recall — did we run the test that would have caught it — is the number that matters. Suite reduction is a side effect, not a goal.

Part 6 • Running the agent

The commands you will actually use, grouped by what you are trying to do.

6.1 Commands by task

I WANT TO...	RUN
Check the install is healthy	<code>python run_poc.py --selftest</code>
See it work without any credentials	<code>python run_poc.py --dry-run</code>
Present it to an audience	<code>python run_poc.py --showcase --pace 1.0</code>
Compare what the two flows select	<code>python run_poc.py --report</code>
Watch the agent self-heal	<code>python run_poc.py --flow 1</code>
See the fix-test vs file-defect split	<code>python run_poc.py --demo</code>
Find untested risky areas	<code>python run_poc.py --coverage</code>
Check my test map is sound	<code>python run_poc.py --check-data</code>
Know how accurate selection is	<code>python run_poc.py --benchmark</code>
Confirm a change did not cause drift	<code>python run_poc.py --eval</code>
Reset to a clean demo state	<code>python run_poc.py --reset</code>

6.2 Schedule profiles

The same agent behaves differently depending on when it runs. A profile changes the lookback window, the threshold and the test budget together.

PROFILE	LOOKBACK	THRESHOLD	TEST BUDGET
pr	14 days	40% of default (2.0)	20 tests — a merge gate must stay fast
daily	30 days	60% of default (3.0)	50 tests
sprint	90 days	the default (5.0)	no cap
quarterly	365 days	40% of default (2.0)	no cap — wide and deep

Run one with `--schedule daily`, or compare all four with `--schedules`. The thresholds are derived from `RISK_THRESHOLD`, so changing that shifts all four profiles together.

6.3 What gets written where

PATH	CONTENTS
<code>._agent_state/</code>	Scores, selections and filed-defect records from the last run. Regenerated every time; safe to delete.
<code>_proposed_fixes/</code>	Proposed test fixes awaiting review, when <code>PR_MODE</code> is off.
<code>benchmark/ground_truth.csv</code>	The executed ground truth, rebuilt by <code>--build-truth</code> .
<code>eval/benchmark_results.json</code>	The most recent accuracy figures.
<code>src/swaglabs.py</code>	The demo application. <code>--flow</code> genuinely edits this; <code>--reset</code> restores it.

All of these except `src/swaglabs.py` are excluded from version control.

Part 7 • Using it on your own project

Moving from the bundled demo to your product, in the order that works.

1. Agree your component list

Ten to thirty names covering your application. Write them down; every other step refers to them. Consistency matters far more than granularity.

2. Write the test map

Replace `tests/test_map.json` with your own test files and their components. Then run `--check-data` and fix what it reports.

3. Point Flow 1 at your defect history

Either export your resolved defects into `data/jira_export.csv` in the shape given in 4.2, or set `JIRA_LIVE=1` with a `JIRA_JQL` that selects your resolved bugs. Verify with `python scripts/check_jira.py`.

4. Generate churn from git

Produce `data/git_churn.csv` from `git log` as a scheduled job, mapping changed paths to component names.

5. Point Flow 2 at your release delta

Either fill the four CSVs in `data/release_delta/`, or set `CHANGE_LIVE=1` and label your issues `release-delta` plus `feature` or `bugfix`. Verify with `python scripts/check_changes.py`.

6. Calibrate before you trust it

Run it against two or three releases you already know the outcome of. Compare what it selected against what actually broke, and adjust `RISK_THRESHOLD`. Do this before letting it gate anything.

7. Decide the human gates

In a pilot, leave `APPROVE_ALL` unset so every gate blocks and a person signs off. Relax it only for the gates your team is comfortable with.

DO NOT SKIP STEP 6

The weights were tuned against a demo application and a sixteen-bug benchmark. They are a starting point, not a calibrated model for your codebase. Selection quality on your project is unknown until you measure it.

Part 8 • Troubleshooting

The failures people actually hit, and what each one means.

SYMPTOM	CAUSE AND FIX
Step 1 says <code>csv</code> when you expected <code>jira (live)</code>	Live mode is off or the credentials are incomplete. The agent falls back deliberately rather than failing. Check <code>JIRA_LIVE=1</code> and all three <code>ATLASSIAN_*</code> values, then run <code>python scripts/check_jira.py</code> .
Second demo run finds nothing to fix	The first run really did repair the source. Run <code>python run_poc.py --reset</code> between runs.
<code>ModuleNotFoundError: anthropic</code>	Packages not installed, or the virtual environment is not active. Re-run <code>python -m pip install -r requirements.txt</code> .
Missing or invalid API key	Only <code>--flow</code> and <code>--demo</code> need one. Use <code>--dry-run</code> or <code>--showcase</code> to demonstrate everything else without a key.
Agent run stops at a gate	Working as designed. Set <code>APPROVE_ALL=1</code> for an uninterrupted demo.
A new Jira ticket appears on every rehearsal	<code>REPORT_BUGS</code> is on. Set it to <code>0</code> while practising.
Defect filing fails on issue type	Your project has no <code>Bug</code> type. Set <code>JIRA_DEFECT_ISSUETYPE=Task</code> .
Everything is selected, or nothing is	The threshold is wrong for your data's scale. Try <code>--threshold</code> at a few values and watch <code>--report</code> .
A component never gets selected	Usually it has no test in <code>tests/test_map.json</code> , or its name is spelled differently there than in your data. Run <code>--check-data</code> .
Confluence requirements not found	Either the page id is wrong, or the rows are not keyed <code>REQ-###</code> . Run <code>python scripts/check_changes.py</code> .
Console shows boxes or garbled symbols	An older terminal without full Unicode. Add <code>--ascii</code> .

START HERE FOR ANYTHING ELSE

`python run_poc.py --selftest` checks dependencies, data files, the test map and live connections in one pass, and names the specific thing that is wrong.

Appendix • Repository map

Where things live, for when you need to read the code itself.

PATH	WHAT IT IS
<code>run_poc.py</code>	The single entry point. Every command in this guide goes through it.
<code>agent/common.py</code>	The shared Reason → Act → Observe loop, the threshold, the model call.
<code>agent/flow1_defect_history.py</code>	Flow 1 tools and the risk model.
<code>agent/flow2_change_driven.py</code>	Flow 2 tools and the impact model.
<code>agent/jira_source.py</code>	Live Jira defect reads.
<code>agent/change_source.py</code>	Live release-delta reads.
<code>agent/confluence_source.py</code>	Live Confluence requirement reads.
<code>agent/jira_report.py</code>	Filing a defect back to Jira.
<code>agent/approvals.py</code>	The four human approval gates.
<code>agent/pr_flow.py</code>	Test fixes delivered as a pull request, never to main.
<code>agent/signals.py</code>	Phase 3 signals: business value, sprint health, RCA, feedback.
<code>agent/arch_risk.py</code>	Architectural risk from code structure.
<code>agent/coverage_gap.py</code>	Risk areas with no test.
<code>agent/schedules.py</code>	The four schedule profiles.
<code>agent/selftest.py</code>	The one-command health check.
<code>agent/showcase.py</code>	The ten-phase console presentation.
<code>benchmark/</code>	The networkx accuracy experiment: seeded bugs, harness, ground truth.
<code>eval/</code>	Scoring: the fixed eval set, the accuracy benchmark, human comparison.
<code>src/swaglabs.py</code>	The demo application under test, with two deliberately planted bugs.
<code>tests/</code>	The demo test suite and <code>test_map.json</code> .
<code>scripts/check_jira.py</code>	Verify the live Jira connection.
<code>scripts/check_changes.py</code>	Verify the live Jira and Confluence connections.
<code>scripts/reset_demo.py</code>	Re-plant the demo bugs and clear state.
<code>seed/</code>	Create the sample issues and pages in a real Atlassian site.
<code>docs/BENCHMARK.md</code>	Method and results of the accuracy experiment.

<code>docs/DEMO_RUNBOOK.md</code>	The presentation script.
<code>CHEATSHEET.md</code>	One-page command reference for demo day.